

Khalasni Mobile Application Proposal

Customer App for Service Browsing, Order Submission, Payment Channel, and Order Tracking

Item	Description
Document Type	Mobile Application Proposal
Platform Scope	Customer mobile app integrated with Django backend
Backend Dependency	Django service ordering website and admin control panel
Target Users	Customers requesting services
Version	1.0

1. Executive Summary

This proposal describes the development of a customer-facing mobile application for Khalasni. The app will allow customers to browse available services, submit service orders, select or indicate a payment method, and track request status.

The mobile app will connect to the Django-based service ordering backend and admin control panel. The backend will remain the source of truth for services, orders, payment records, and reports.

This version is intentionally focused. It does not include provider apps, wallet systems, full accounting, automatic dispatch, or advanced marketplace features.

2. Project Objective

The objective is to make service requesting easier for customers while allowing the business team to manage services, orders, payments, and reports from the Django control panel.

- Provide a clean customer mobile interface.
- Show active services from the backend.
- Allow customers to submit service orders.
- Allow customers to select or view payment channels.
- Allow customers to check basic order status.
- Keep the app simple, reliable, and ready for future phases.

3. Proposed System Overview

Part	Purpose
Customer Mobile App	Used by customers to browse services and submit orders.
Django Backend API	Provides services, order creation, payment method data, and order status.
Admin Control Panel	Used by the business team to manage services, orders, payments, and reports.

4. Core Mobile App Modules

Module	Description
Splash and Onboarding	Branded loading screen and optional short introduction screens.
Home Screen	Service categories, featured services, banners, search/filter option, and contact shortcut.
Service Browsing	Display active service categories and services managed from the Django admin panel.
Service Details	Show description, image, price if applicable, instructions, and order button.
Order Submission	Customer enters name, phone, address, requested date/time, and notes, then submits the order.

Payment Channel	Customer chooses or views available payment channels such as Cash, CliQ, Bank Transfer, Card, or Other.
Order Tracking	Customer views submitted orders, order details, status, and payment status if enabled.
Notifications	Optional order submitted, confirmed, completed, or payment-status notifications.
Customer Profile	Optional customer name, phone, saved address, and order history if login is implemented.

5. Recommended Technical Approach

The recommended mobile technology is Flutter because it allows building Android and iOS applications from one codebase. The first operational release may focus on Android if required.

Technology	Role
Flutter	Customer mobile application.
Django REST API	Backend communication layer.
PostgreSQL	Backend database.
Django Admin	Internal business control panel.

If the client wants the lowest-maintenance option before a native app, a Progressive Web App can be considered. However, Flutter is the stronger choice when a real installable mobile application is required.

6. Backend Integration

The mobile app will communicate with the Django backend through API endpoints. The backend controls services, orders, payment records, and status updates.

API Area	Function
Services API	Return active service categories and services.
Order API	Create new service orders from the mobile app.
Payment API	Return available payment methods and basic payment fields.
Order Tracking API	Return customer order status.
Content API	Return banners, contact details, and basic app content if needed.

7. Main User Flows

Flow	Steps
Customer Service Request	Open app -> browse services -> select service -> fill order form -> choose payment method -> submit order -> receive confirmation.
Order Tracking	Open app -> enter phone/login -> view submitted orders -> open order details -> check status.
Admin Management	Admin edits services in Django -> app reflects active services -> customer submits order -> admin updates status -> app shows latest status.

8. Scope of Work

8.1 Included in the First Version

- Mobile app UI based on Khalasni branding.
- Home, service categories, service list, and service details screens.
- Order submission form.
- Payment method/channel screen.
- Order confirmation screen.
- Basic order tracking.
- Django backend API integration.
- Basic validation and error handling.
- Android build preparation.
- Basic handover documentation.

8.2 Optional Features

- Customer login and saved profile.
- Saved addresses.
- Push notifications.
- Receipt upload.
- Online payment gateway.
- Arabic and English support.
- iOS build preparation.
- App store publishing support.

9. Out of Scope for This Version

- Provider mobile application.
- Live technician/driver tracking.
- Automatic dispatch.
- Wallet system.
- Provider payout system.
- Full accounting system.
- Real-time chat.
- Advanced CRM.
- AI-based matching.
- Full marketplace functionality.

10. Proposed Screens

Screen	Description
Splash Screen	Initial branded loading screen.
Home Screen	Categories, banners, and quick service access.
Service Categories	Main service categories.
Services List	Services under selected category.
Service Details	Description, image, price if available, and order button.
Order Form	Customer details, address, notes, and preferred date/time.
Payment Method	Available payment options and instructions.
Order Success	Successful order confirmation.
My Orders	Submitted orders and current status.
Order Details	Order summary, status, and payment information.
Contact/Support	Phone, WhatsApp, and support information.

11. Data Handled by the Mobile App

- Customer name.
- Customer phone number.
- Customer address or location description.
- Selected service.
- Requested date/time if required.
- Customer notes.
- Selected payment method.
- Payment reference if enabled.

Sensitive data should be minimized. The backend should remain the main source of truth for orders, payment status, and reporting.

12. Security and Reliability Considerations

- Use HTTPS for all API communication.
- Validate all customer input on the backend.
- Do not trust mobile-side pricing or status values.
- Keep payment status controlled by the backend/admin panel.
- Use secure token storage if login is implemented.
- Limit uploaded file types and sizes if receipt upload is added.
- Handle network errors clearly.
- Avoid exposing admin functions inside the customer app.

13. Future Expansion Possibilities

- Provider app.
- Customer account system.
- Online payment gateway.
- Push notifications.
- Live order updates.
- Coupons and promotions.
- Ratings and reviews.
- Complaint submission.
- Multi-language interface.
- Map/location integration.
- External accounting integration.

14. Expected Deliverables

- Flutter mobile application source code.
- Integrated API communication with Django backend.
- Customer service browsing screens.
- Order submission flow.
- Payment method/channel screen.
- Basic order tracking flow.
- Android application build file.
- Basic technical documentation.
- Basic user guide.
- Build instructions.

15. Success Criteria

- Customer can open the app and view active services.
- Customer can submit a service order successfully.
- Submitted orders appear in the Django admin control panel.
- Customer can choose or view payment method/channel.
- Admin can update order status from the backend.
- Customer can view basic order status from the app.
- The app is stable, clear, and ready for future expansion.

16. Final Recommendation

The recommended solution is to develop a focused customer mobile app that connects to the Django-based service ordering system. The app should cover service browsing, order submission, payment method display, and order tracking, while the admin control panel remains in Django.

This approach keeps the mobile app simple and useful while avoiding unnecessary complexity. It also keeps the system ready for future phases such as customer accounts, provider apps, online payments, notifications, and advanced tracking.